

CSC 402 Lab 2: Kotlin & Jetpack Compose Fundamentals

Student: Narjes Al-Wohaib

ID: 2240002170

Major: Computer Science

Level: 7

Part A: Kotlin Console Tasks

Task 1 Output

```
--- Task 1: Student Information ---  
Name: Narjes Al-Wohaib  
ID: 2240002170  
GPA: 4.42  
Credits: 78  
Status: Active
```

Task 3 Output

```
--- Task 3: Score Calculation ---  
Student Narjes achieved a weighted score of 91.40
```

Task 4 Output

```
--- Task 4: List of Languages ---  
Original list: [Kotlin, Java, Python, C++, Swift]  
First language: Kotlin  
Last language: Swift  
Sorted list: [C++, Java, Kotlin, Python, Swift]  
Does the list contain Kotlin? Yes
```

Task 5 Output

```
--- Task 5: Course Management ---  
Courses with >= 3 credits: [CSC 402, CSC 311, CSC 340, MATH 202]  
All course titles: [Mobile Application Programming, Database Systems, Operating Systems]  
Total registered credits: 15  
Search Result (CSC 402): Mobile Application Programming  
Updated Course Schedule: CSC 402 - Mon / Wed / Fri
```

Part B: Jetpack Compose Tasks

Task 6 — Welcome Screen

1:01 

CSC 402 Lab 2

Task 6 - My First Screen

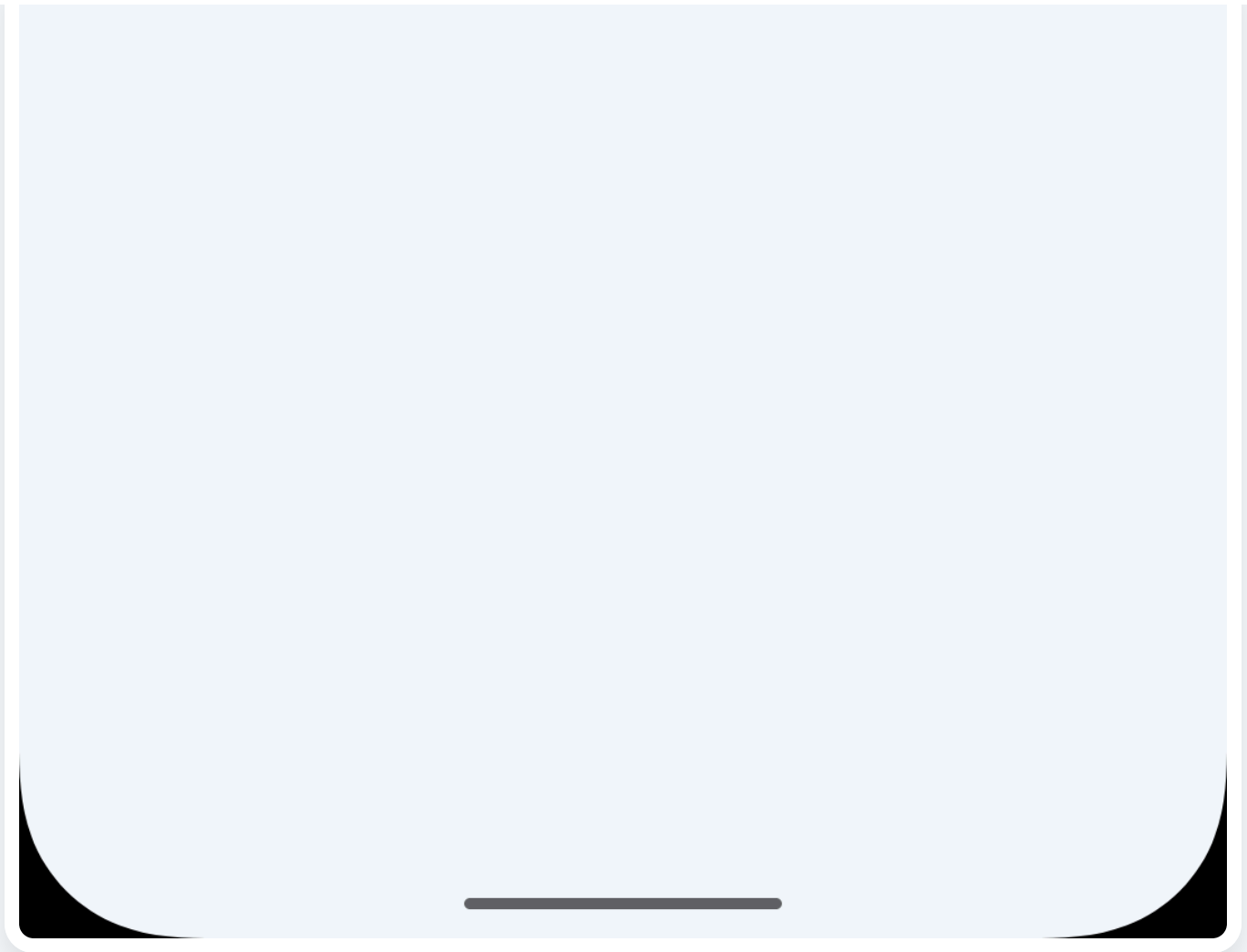


Hello, Narjes!

Welcome to Mobile Application
Programming

Built with Jetpack Compose

Everything on this screen is drawn
by one composable function.



Task 7 — Student ID Screen

1:03



CSC 402 Lab 2

Task 7 - Student Identity

NA

Narjes Al-Wohaib

Computer Science

ID: 2240002170

Level 7

CSHJ

4.42

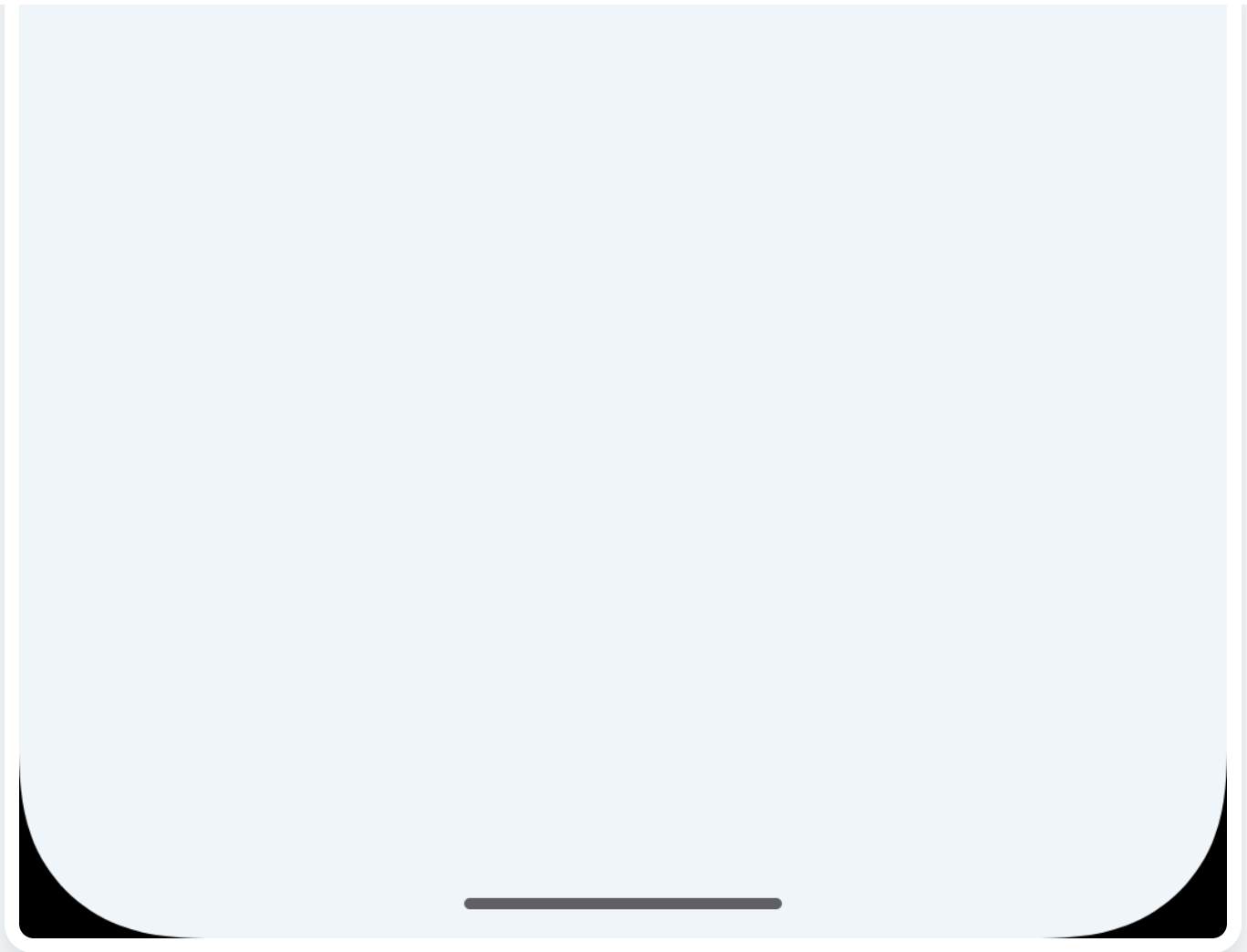
GPA

78

Credits

Active

Status



Task 8 — Counter Screen

1:05



CSC 402 Lab 2

Task 8 - State Management

Current Value

0

Even Number

-

+

Reset Counter

Task 9 — Greeting Screen

1:06



CSC 402 Lab 2

Task 9 - Text Input & State

Welcome, Narjes!

Letters entered: 6

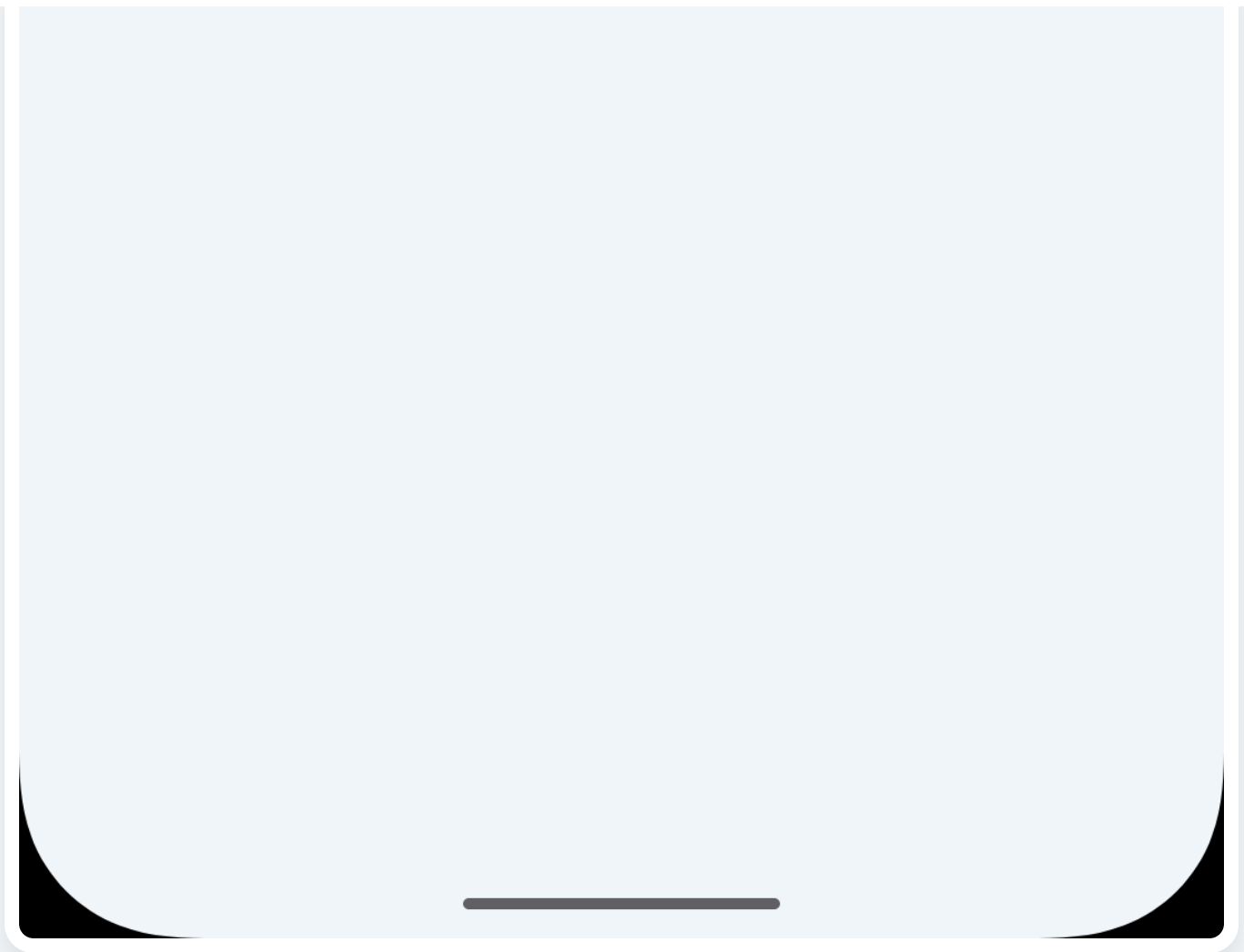
Your Name

Narjes

Clear

Experiment: The name variable is updated on every keystroke, causing the UI to recompose and update the text and count instantly.





Task 10 — Course List Screen

1:06



CSC 402 Lab 2

Task 10 - Lists & LazyColumn

CSC 402**Mobile Application Programming**

Mon / Wed

3 Cr

CSC 311**Database Systems**

Sun / Tue

4 Cr

CSC 340**Operating Systems**

Mon / Wed

3 Cr

MATH 202**Discrete Mathematics**

Sun / Thu

3 Cr

ENG 214**Technical Writing**

Tue

2 Cr

CSC 202

Data Structures and Algorithms

Sun / Tue

3 Cr

CSC 330

Software Engineering Principles

Mon / Wed

3 Cr

Task 11 — Task Tracker Screen

1:07



CSC 402 Lab 2

Task 11 - State Hosting & Lists

2

Done

2

Open

4

Total

**~~Fix navigation bug~~**

Owner: Narjes Al-Wohaib

**~~Update project theme~~**

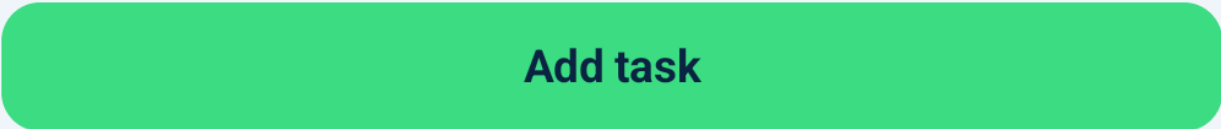
Owner: Narjes Al-Wohaib

**Implement course list**

Owner: Narjes Al-Wohaib

**Write lab report**

Owner: Narjes Al-Wohaib



Add task

Questions & Answers

Q1: What is the main difference between 'val' and 'var' in Kotlin?

'val' is used for immutable references, meaning the variable cannot be reassigned once initialized. 'var' is for mutable variables that can change their value throughout the program's execution.

Q2: Why are data classes useful in Kotlin application development?

Data classes automatically generate essential methods like `equals()`, `hashCode()`, and `toString()` based on the properties defined in the constructor. This significantly reduces boilerplate code for classes that primarily hold data.

Q3: How does the 'remember' function help maintain UI state during recomposition?

'remember' caches a value across recompositions. Without it, local variables would be reset every time the composable function is re-executed, losing the user's current state (like a counter value).

Q4: Explain the role of 'mutableStateOf' in Jetpack Compose.

'mutableStateOf' creates an observable State object. When its value changes, Compose automatically identifies which parts of the UI depend on that value and triggers a recomposition to update them.

Q5: When should you use 'LazyColumn' instead of a standard 'Column'?

'LazyColumn' should be used when displaying a large number of items or a list of unknown length. It only composes and lays out items that are currently visible, which is much more efficient than a 'Column' that renders everything at once.

Q6: What is 'State Hoisting' and why is it recommended?

State hoisting is a pattern where state is moved to a component's caller to make the component stateless. This improves reusability, makes testing easier, and allows for a single source of truth managed by the parent.

Q7: Describe a scenario where you would use a 'derived state' in a Compose screen.

Derived state is useful when a value is calculated based on other state variables. For example, in the Task Tracker, the count of 'Done' tasks is a derived state that automatically updates whenever the task list changes.

Q8: How does 'Modifier.weight(1f)' affect the layout within a Row or Column?

'Modifier.weight' allows an element to take up a proportional amount of the remaining space in a Row or Column. A weight of 1f means it will expand to fill the available space after other non-weighted elements are placed.

AI Use & Disclosure Log

I utilized Android Studio Gemini and ChatGPT to assist with the structured implementation of Material 3 components, specifically for the TaskTracker list logic and styling the Student ID card. These tools helped ensure consistent design patterns across all six screens.

CSC 402 - Mobile Application Programming | Lab 2 Submission